

1. Obiective și Arhitectură

Am implementat stratul de orchestrare a datelor (Pipeline) care asigură fluxul informațional între serviciul de extracție și sistemele de persistență. Proiectul respectă arhitectura stratificată (Controller -> Service -> Repository).

Componente cheie implementate:

- Pipeline Controller: Expune endpoint-ul `POST /api/v1/pipeline/process` pentru declanșarea fluxului ETL.
- Extraction Client: Un client HTTP bazat pe `RestTemplate` (configurat cu timeout de 5s) care comunică cu Modulul 1 pentru obținerea datelor structurate.
- Pipeline Service: Coordonează logica de business (mapare DTO -> Entity, salvare tranzacțională în SQL și trimitere către Vector Store).
- Dual Persistence Layer: Integrare cu PostgreSQL (JPA) și interfațarea cu Vector Store-ul (pregătit pentru integrarea cu Qdrant în M2).

2. Metodologie de Dezvoltare (Utilizare Gemini CLI v0.35.3)

Pentru eficientizarea procesului și eliminarea erorilor de configurare Maven/Java 21, am utilizat **Gemini CLI** ca partener de programare.

Task-uri automatizate prin prompt-uri tehnice:

1. Configurare Mediu: Generarea structurii proiectului (mvnw, pom.xml) cu dependențele corecte (Spring Data JPA, PostgreSQL, Lombok).
2. Generare Modele (DTO/Entity): Crearea record-urilor `RawInputDTO`, `ExtractedExpenseDTO` și a entității `ExpenseEntity` cu adnotări `@Builder` și `@Data` pentru mapare rapidă.
3. Implementare Client HTTP: Configurarea `RestTemplateConfig` și a logicii de consum API pentru Modulul 1.
4. Definire Contracte Integrare: Generarea interfeței `ExpenseVectorRepository` pentru a permite echipei AI (M2) să implementeze stocarea vectorială fără a bloca pipeline-ul principal.

3. Livrabile Tehnice (Fisiere pe branch-ul teams/M3)

- Config: `RestTemplateConfig.java`, `pom.xml`
- Client: `ExtractionClient.java`
- Service: `ExpensePipelineService.java`
- Model/DTO: `ExpenseEntity.java`, `ExtractedExpenseDTO.java`, `RawInputDTO.java`
- Repository: `ExpenseJpaRepository.java`, `ExpenseVectorRepository.java`

4. Metodologie de Dezvoltare: Utilizarea Gemini CLI (v0.35.3)

Am utilizat Gemini CLI ca partener de tip *pair-programming* pentru a asigura o arhitectură curată și pentru a elimina erorile de configurare Maven/Java 21. Mai jos sunt detaliate prompturile principale care au construit Modulul 3 (Integration & Data Pipeline).

1. Configurarea Mediului de Build (Infrastructură)

- Prompt: *"Sunt Laura, responsabila de Modulul 3 conform GEMINI.md. Inițializează structura proiectului în C:\ProiectulMeuMare. Generează wrapper-ul Maven (mvnw, mvnw.cmd) și configurează un pom.xml pentru Java 21. Adaugă dependențele: Spring Web, JPA, PostgreSQL și Lombok."*
- Rezultat: pom.xml, mvnw, .mvn/wrapper.

2. Definirea Structurii de Date (Modele și DTO-uri)

- Prompt: *"Generează în pachetul dto record-urile RawInputDTO și ExtractedExpenseDTO. Apoi, creează ExpenseEntity.java în pachetul model cu câmpurile: amount, currency, category și transactionDate. Folosește adnotările Lombok @Data, @Builder și @AllArgsConstructor."*
- Rezultat: RawInputDTO.java, ExtractedExpenseDTO.java, ExpenseEntity.java.

3. Integrarea cu Modulul 1 (Client HTTP)

- Prompt: *"Configurează un RestTemplate bean în RestTemplateConfig.java cu un timeout de 5 secunde. Creează ExtractionClient.java care să trimită RawInputDTO către endpoint-ul Dumitriței și să primească datele structurate."*
- Rezultat: RestTemplateConfig.java, ExtractionClient.java.

4. Definirea Contractelor de Persistență (SQL & Vector Store)

- Prompt: *"Generează ExpenseJpaRepository pentru PostgreSQL. Creează și interfața ExpenseVectorRepository cu metodele saveVector și existsInVectorStore pentru a permite integrarea ulterioară cu Qdrant a Alexiei (M2)."*
- Rezultat: ExpenseJpaRepository.java, ExpenseVectorRepository.java.

5. Orchestrarea Fluxului (Pipeline & Controller)

- Prompt: *"Scrie ExpensePipelineService.java. Metoda trebuie să fie @Transactional și să urmeze pașii: 1. Extract (M1), 2. Save SQL, 3. Generare dummy vector, 4. Save Vector (M2). Adaugă și un PipelineController pentru declanșare via POST."*
- Rezultat: ExpensePipelineService.java, PipelineController.java.

Fluxul de Date (Pipeline M3)

1. Input: Clientul trimite text brut (ex: "100 lei la Lidl") către `PipelineController`.
2. Processing: `ExpensePipelineService` apelează `ExtractionClient` pentru a obține datele procesate.
3. Storage: Datele sunt salvate în PostgreSQL și un vector de dimensiune 10 este trimis către Vector Store.

Avantaje	Dezavantaje
Viteză de Setup: A generat instant <code>pom.xml</code> și <code>mvnw</code> , eliminând erorile de configurare Java 21.	Precizia Promptului: Dacă nu ești specific, AI-ul poate genera cod care nu se aliniază cu restul echipei.
Context Local: Știe structura folderelor și pune fișierele direct unde trebuie (ex: în <code>com.proiect.service</code>).	Control Manual: Necesită atenție sporită la <code>git status</code> pentru a nu lăsa AI-ul să suprascrie munca ta.
Arhitectură Corectă: Respectă automat straturile (Controller -> Service -> Repository) fără efort manual.	Lipsa Interfeței: Lucrul doar în terminal poate fi obositor față de o interfață grafică (web/IDE).